

Contents

1	odbfiler User Guide	3
1.1	Overview	3
1.1.1	Key Features	4
1.2	Installation	4
1.2.1	Prerequisites	4
1.2.2	Build with CMake (Recommended)	4
1.2.3	Build in a Pack Environment	4
1.2.4	Build Options	4
1.3	Quick Start	5
1.3.1	Basic Usage	5
1.3.2	Filter Types	5
1.4	Common Use Cases	6
1.4.1	Disable All Observations Except One TEMP Temperature	6
1.4.2	Keep Only TEMP Observations	6
1.4.3	Delete All TEMP Observations	6
1.4.4	Select Observations for a Specific Time Slot	6
1.4.5	Filter by Geographic Area	6
1.5	Using Wrapper Scripts	7
1.5.1	odb_filter.sh	7
1.5.2	odbtbx.sh	7
1.5.3	CCMA2ECMA	7
1.6	Adding Custom SQL Filters	7
1.7	Troubleshooting	8
1.7.1	“odb98.x not found”	8
1.7.2	“ODB library not found”	8
1.7.3	Compilation errors with generated C files	8
2	odbfiler Developer Guide	8
2.1	Project Structure	8
2.2	Build System	9
2.2.1	Overview	9
2.2.2	CMakeLists.txt Structure	9
2.2.3	Adding a New Schema	10
2.2.4	Template System	10
2.2.5	Generated Files	10
2.3	Code Architecture	10
2.3.1	Main Program (src/filter.F90)	10
2.3.2	Module (src/odb_filter_mod.F90)	11
2.3.3	C Glue Code	11
2.4	Adding New Features	11
2.4.1	Adding a New Filter Type	11
2.4.2	Adding a New SQL View	12
2.4.3	Modifying Templates	12
2.5	Build Options	12

2.5.1	CMake Variables	12
2.5.2	Compilation Flags	12
2.6	Dependencies	13
2.6.1	Required	13
2.6.2	Optional	13
2.7	Testing	13
2.7.1	Manual Testing	13
2.7.2	Adding New Tests	13
2.8	Contributing	13
2.8.1	Code Style	13
2.8.2	Commit Messages	13
2.9	Debugging	14
2.9.1	Common Issues	14
2.9.2	Debug Output	14
3	odbfiler Technical Reference	14
3.1	ODB (Observations Data Base)	14
3.1.1	Overview	14
3.1.2	Key Concepts	14
3.1.3	ECMA/CCMA Schema	14
3.2	SQL Files	15
3.2.1	Location	15
3.2.2	File Structure	15
3.2.3	Key Keywords	15
3.2.4	Available SQL Files	15
3.3	ODB Macros and Functions	17
3.3.1	In C Code (Generated)	17
3.3.2	ODB API Functions	17
3.4	Build Integration	18
3.4.1	odb98.x SQL Compiler	18
3.4.2	Generated Files	18
3.4.3	Object Libraries	18
3.5	Environment Variables	19
3.5.1	ODB Runtime Variables	19
3.5.2	Build Environment Variables	19
3.6	Database Tables	19
3.6.1	hdr (Header Table)	19
3.6.2	body (Body Table)	20
3.6.3	errstat (Error Statistics Table)	20
3.7	Fortran-ODB Integration	20
3.7.1	Key Modules and Functions	20
3.7.2	Special Values	21
3.8	Troubleshooting	21
3.8.1	SQL Compilation Errors	21
3.8.2	Missing Headers	21
3.8.3	Schema Not Found	21

4	odbfiler Examples	21
4.1	Command-Line Usage	21
4.1.1	Basic Syntax	21
4.2	Filter Examples	22
4.2.1	1. Select Observations by Time Slot	22
4.2.2	2. Keep Only Specific Observation Type	22
4.2.3	3. Delete Specific Observation Type	22
4.2.4	4. Filter by Observation Type and Variable	22
4.2.5	5. Select Single Observation	22
4.2.6	6. Select Wind Observations	23
4.2.7	7. Filter by Geographic Area	23
4.2.8	8. Filter by Station ID	23
4.2.9	9. Filter Scatterometer Observations	24
4.2.10	10. Filter by Sensor	24
4.2.11	11. Select Single Radar Observation	24
4.2.12	12. Select Single GPS Solution	25
4.2.13	13. Filter by Channel (Canal)	25
4.2.14	14. Copy First Guess Departures	25
4.2.15	15. Copy Analysis Departures	25
4.2.16	16. Delete Passive Observations	26
4.2.17	17. Debug Filter	26
4.3	Using Wrapper Scripts	26
4.3.1	odb_filter.sh	26
4.3.2	odbtbx.sh (Toolbox)	26
4.3.3	CCMA2ECMA	27
4.4	Advanced Scenarios	27
4.4.1	Batch Processing Multiple Filters	27
4.4.2	Combining Filters	27
4.4.3	Working with Wrapper Scripts in Production	27
4.5	Verifying Results	27
4.5.1	Check Active Observations	27
4.5.2	Check Report Status	28
4.6	Common Patterns	28
4.6.1	Keep Only One Observation Type	28
4.6.2	Select Specific Observations	28
4.6.3	Filter by Area	28
4.6.4	Work with Wind	28

1 odbfiler User Guide

1.1 Overview

odbfiler is a tool for filtering and updating ODB (Observations Data Base) databases used in numerical weather prediction. It allows you to selectively activate or deactivate observations based on customizable SQL requests.

1.1.1 Key Features

- Filter observations by type, time slot, geographic area, and more
 - Works with CCMA and ECMA database schemas
 - Customizable SQL views for complex filtering logic
 - Integration with ODB shuffle operations (CCMA <-> ECMA)
-

1.2 Installation

1.2.1 Prerequisites

- **ODB libraries** (libcma-odb, libodb, libmain-odb, libifsaux) - typically from a pack
- **NetCDF** (C and Fortran libraries)
- **Fortran compiler** (gfortran or Intel ifort)
- **C compiler** (gcc or Intel icc)
- **CMake** (version 3.10 or higher)
- **odb98.x** (ODB SQL compiler, from ODB installation)

1.2.2 Build with CMake (Recommended)

```
cd odbfilter
mkdir build && cd build
cmake .. \
  -DLIBNCDF=/path/to/netcdf \
  -DLIBNCDF_F=/path/to/netcdf_f
make
make install
```

1.2.3 Build in a Pack Environment

For IAL gmkpack environments (CY49T1, CY50T2, etc.), use the provided build script:

```
./build_for_pack.sh
```

This script: - Loads the correct environment modules - Extracts NetCDF paths from pack configuration - Configures CMake with appropriate compiler and library paths - Builds and installs the project

1.2.4 Build Options

Option	Description	Default
-DPACK=	Pack root directory	\${CMAKE_SOURCE_DIR}
-DTYPE_PACK=	Pack type: “main” (production) or “local” (development)	“main”

Option	Description	Default
-DLIBNCDF=	NetCDF C library path	“”
-DLIBNCDF=	NetCDF Fortran library path	“”
-DFIAT_DIR=	Fiat library installation directory (optional)	“”

1.3 Quick Start

1.3.1 Basic Usage

`odb_filter <filter_type> [arguments]`

The `odb_filter` executable reads from a CCMA database and modifies observation statuses based on the specified filter type.

1.3.2 Filter Types

Filter Type	Description	Arguments
<code>timeslot</code>	Select observations for a specific time slot	<code>timeslot</code>
<code>type</code>	Keep only specified observation type	<code>obstype</code>
<code>deletetype</code>	Delete specified observation type	<code>obstype</code>
<code>varno</code>	Filter by observation type and variable number	<code>obstype, varno</code>
<code>wind</code>	Filter wind observations (u/v components)	<code>obstype, varno_u, timeslot, partition</code>
<code>singleobs</code>	Select single observation of specified type/varno	<code>obstype, varno, [timeslot], [nb_obs]</code>
<code>singlewind</code>	Select single wind observation	<code>obstype, varno_u, timeslot, [nb_obs]</code>
<code>scat</code>	Filter scatterometer observations	<code>timeslot, [nb_obs]</code>
<code>sensor</code>	Filter by observation type and sensor	<code>obstype, sensor</code>
<code>singleradar</code>	Select single radar observation	<code>obstype, varno, ident, distance, elev, azimuth</code>
<code>singlegpsol</code>	Select single GPS sol observation	<code>obstype, codetype, lon, lat</code>
<code>singlestatid</code>	Filter by station ID	<code>statid</code>

Filter Type	Description	Arguments
singleobsarea	Select observation in geographic area	obstype, varno, lonmin/max, latmin/max, presmin/max
singleobsanal	Filter by channel (canal)	obstype, vertco
sophie	Custom selection for Sophie	-
fg_depar	Copy first guess departures	-
an_depar	Copy analysis departures	-
delete_passive	Delete passive observations	-
debug	Debug filter	-

1.4 Common Use Cases

1.4.1 Disable All Observations Except One TEMP Temperature

```
odb_filter singleobs 5 2
```

This selects only TEMP temperature observations (obstype=5, varno=2).

1.4.2 Keep Only TEMP Observations

```
odb_filter type 5
```

Marks all non-TEMP observations as passive.

1.4.3 Delete All TEMP Observations

```
odb_filter deletetype 5
```

Removes all TEMP observations from the database.

1.4.4 Select Observations for a Specific Time Slot

```
odb_filter timeslot 12
```

Keeps only observations from time slot 12.

1.4.5 Filter by Geographic Area

```
odb_filter singleobsarea 5 2 -10 10 40 50 950 1050
```

Selects observations of type 5, varno 2 within: - Longitude: -10 to 10 - Latitude: 40 to 50 - Pressure: 950 to 1050 hPa

1.5 Using Wrapper Scripts

1.5.1 odb_filter.sh

Wrapper script that sets up the ODB environment and runs odb_filter:

```
scripts/odb_filter.sh <filter_type> [arguments]
```

This script also performs a shuffle operation (CCMA → ECMA) and copies first guess departures.

1.5.2 odbtbx.sh

Toolbox script providing filter and shuffle functions:

```
# Apply filter
```

```
scripts/odbtbx.sh -f <filter_type> [arguments]
```

```
# Perform shuffle
```

```
scripts/odbtbx.sh -s
```

1.5.3 CCMA2ECMA

Converts a CCMA database to a pseudo-ECMA database:

```
scripts/CCMA2ECMA
```

This is useful for converting conventional observation data to model equivalent format.

1.6 Adding Custom SQL Filters

To add a new filter:

1. Create a new SQL file in `sql/` (e.g., `my_filter.sql`)
2. Define your SQL view following the existing pattern
3. Re-run CMake to auto-detect the new SQL file
4. Modify `src/filter.F90` and `src/odb_filter_mod.F90` to use your new filter

Example SQL file (`sql/my_filter.sql`):

```
//  
/-- ODB/SQL file 'my_filter.sql'  
//
```

```
UPDATED;
```

```

SET $my_param = 100;

CREATE VIEW my_filter AS
  SELECT obsvalue UPDATED
  FROM   hdr, body, errstat
  WHERE  varno = $my_param;

```

1.7 Troubleshooting

1.7.1 “odb98.x not found”

Ensure ODB is installed and odb98.x is in your PATH, or set `-DODB_COMPILER=/path/to/odb98.x`.

1.7.2 “ODB library not found”

Check that your pack environment is correctly set up, or specify library paths manually with `-DPACK=/path/to/pack`.

1.7.3 Compilation errors with generated C files

The C files (`_odb_glue.c`, `*_Sstatic.c`) are auto-generated in the build directory. If you encounter issues: 1. Clean the build: `rm -rf build && mkdir build` 2. Re-run CMake: `cd build && cmake ..` 3. Check that all `ddl.*` directories are correctly set up

2 odbfilter Developer Guide

2.1 Project Structure

```

odbfilter/
+-- CMakeLists.txt           # Main CMake build configuration
+-- build_for_pack.sh        # Build script for pack environments
+-- README.md                # Project overview and quick start
+-- cmake/
|   +-- templates/
|       +-- _odb_glue.c.in    # Template for ODB glue code
|       +-- _Sstatic.c.in     # Template for schema static initializers
+-- src/
|   +-- filter.F90            # Main Fortran program
|   +-- odb_filter_mod.F90    # Fortran module (init/cleanup)
|   +-- _odb_glue.c           # Generated: ODB entry point (in build/)
|   +-- *_Sstatic.c           # Generated: Schema initializers (in build/)
+-- sql/                     # SQL view definitions

```



```

|   +-- select_all.sql
|   +-- select_obstype.sql
|   +-- ... (23 SQL files total)
+-- ddl.CCMA -> sql/           # Schema symlink (auto-detected)
+-- ddl.ECMA -> sql/           # Schema symlink (auto-detected)
+-- scripts/
|   +-- odb_filter.sh         # Wrapper script with ODB environment
|   +-- odbtbx.sh             # Toolbox: filter + shuffle
|   +-- CCMA2ECMA             # Convert CCMA to ECMA format
|   +-- create_ioassign       # Create IOASSIGN file
+-- doc/                      # Documentation (this directory)
|   +-- user-guide.md
|   +-- developer-guide.md    # (this file)
|   +-- technical-reference.md
|   +-- examples.md
+-- build/                    # Build directory (generated)

```

2.2 Build System

2.2.1 Overview

The project uses CMake (minimum version 3.10) with a custom build process:

1. **Auto-detection:** Scans for `ddl.*` directories to identify database schemas
2. **Code Generation:** Uses `configure_file()` with templates in `cmake/templates/`
3. **SQL Processing:** Compiles `.sql` files to `.o` using `odb98.x`
4. **Linking:** Links against ODB libraries and dependencies

2.2.2 CMakeLists.txt Structure

The main `CMakeLists.txt` is organized into these sections:

1. Project setup and options
2. Schema auto-detection (`ddl.*` directories)
3. SQL file discovery
4. Find dependencies (NetCDF, ODB, Fiat)
5. Set include directories
6. Generate C files from templates
7. Add schema subdirectories (`sql/CMakeLists.txt`)
8. Set compilation flags
9. Create object libraries
10. Main executable target
11. Link libraries
12. Installation rules

2.2.3 Adding a New Schema

To add support for a new ODB schema (e.g., “TEST”):

```
cd /home/arbogaste/git/odbfilter
ln -s sql ddl.TEST
rm -rf build && mkdir build && cd build && cmake ..
```

CMake will automatically: - Detect `ddl.TEST` in the glob scan - Add “TEST” to the `_SCHEMAS` list - Generate `build/src/TEST_Sstatic.c` from template - Add `TEST_lib` target from SQL processing - Update `_odb_glue.c` with `Static_Init(TEST);`

2.2.4 Template System

2.2.4.1 `cmake/templates/_odb_glue.c.in` This template generates the ODB entry point code. Key placeholders:

- `@STATIC_INIT_CALLS@`: Replaced with `Static_Init(CCMA);, Static_Init(ECMA);,` etc.

2.2.4.2 `cmake/templates/_Sstatic.c.in` This template generates schema-specific static initializers. Key placeholders:

- `@SCHEMA@`: Replaced with schema name (CCMA, ECMA, etc.)
- `@ODB_ANCHOR_VIEWS@`: Replaced with `ODB_ANCHOR_VIEW()` calls for each SQL view

2.2.5 Generated Files

All generated C files are placed in `${CMAKE_BINARY_DIR}/src/` (not the source directory):

- `_odb_glue.c`: ODB entry point with `codb_set_entrypoint_()`
- `CCMA_Sstatic.c`: CCMA schema initializer
- `ECMA_Sstatic.c`: ECMA schema initializer

This keeps the source directory clean and allows out-of-source builds.

2.3 Code Architecture

2.3.1 Main Program (`src/filter.F90`)

The main Fortran program that: 1. Parses command-line arguments to determine filter type 2. Initializes ODB environment via `odb_filter_start()` 3. Routes to appropriate filter logic (select case) 4. Reads/modifies/writes observations 5. Cleans up via `odb_filter_end()`

Supported Filters (18 total): - timeslot, type, deletetype, varno, wind
 - singleobs, singlewind, scat, sensor - singleradar, singlegpssol,
 singlestatid - singleobsarea, singleobsanal, sophie - fg_depar,
 an_depar, delete_passive, debug

2.3.2 Module (src/odb_filter_mod.F90)

Provides initialization and cleanup routines:

- **odb_filter_start()**: Sets up ODB environment, parses arguments, opens database
- **odb_filter_end()**: Updates header status, closes database
- **odb_env()**: Configures ODB environment variables

2.3.3 C Glue Code

2.3.3.1 _odb_glue.c (Generated) Provides the interface between ODB library and odbfilter:

- **codb_set_entrypoint_()**: Called by ODB to register schema initializers
- **Static_Init(db) macro**: Registers a schema's **db_static_init()** function
- **ctxprint_()**: Dummy callback for ODB

2.3.3.2 *_Sstatic.c (Generated per schema) Schema-specific static initializers:

- **CCMA_static_init()**: Registers all CCMA SQL views
- **ECMA_static_init()**: Registers all ECMA SQL views
- Uses **ODB_ANCHOR()** and **ODB_ANCHOR_VIEW()** macros

2.4 Adding New Features

2.4.1 Adding a New Filter Type

1. Create SQL file in sql/:

```
//
//-- ODB/SQL file 'my_filter.sql'
//
UPDATED;
SET $my_param = 100;
CREATE VIEW my_filter AS
  SELECT obsvalue UPDATED
  FROM   hdr, body
  WHERE  varno = $my_param;
```

2. Update src/odb_filter_mod.F90:

- Add variable: `$my_param`
- Parse argument in `odb_filter_start()`

3. Update `src/filter.F90`:

- Add case in select case: `case ('myfilter')`
- Implement filter logic

4. Rebuild:

```
cd build && cmake .. && make
```

2.4.2 Adding a New SQL View

Simply add a new `.sql` file to `sql/`. CMake will: - Auto-detect the new file - Add `ODB_ANCHOR_VIEW()` call to each schema's `_Sstatic.c`

2.4.3 Modifying Templates

To change the generated C code:

1. Edit `cmake/templates/_odb_glue.c.in` or `_Sstatic.c.in`
2. Rebuild: `cd build && cmake .. && make`

2.5 Build Options

2.5.1 CMake Variables

Variable	Description	Default
PACK	Pack root directory	<code>\${CMAKE_SOURCE_DIR}</code>
TYPE_PACK	“main” (production) or “local” (dev)	“main”
ODB_COMPILER	Path to <code>odb98.x</code>	<code>\${PACK}/sys</code>
INC_CMA	CMA include directory	<code>\${PACK}/src/\${TYPE_PACK}/odb/ddl</code>
INC_ODB	ODB module directory	<code>\${PACK}/src/\${TYPE_PACK}/odb/module</code>
LIBNCDF	NetCDF C library path	“”
LIBNCDF_F	NetCDF Fortran library path	“”
FIAT_DIR	Fiat library installation directory	“”

2.5.2 Compilation Flags

- `-DWITHOUT_OML`: Compiled into all C files (disables Ocean Model Library support)
- **Fortran flags**: Also use `-DWITHOUT_OML`

2.6 Dependencies

2.6.1 Required

- **ODB Libraries:** libcma-odb, libodb, libmain-odb, libifsaux
- **odb98.x:** ODB SQL compiler (from ODB installation)
- **NetCDF:** libnetcdf, libnetcdff
- **Compilers:** C (gcc/icc), Fortran (gfortran/ifort)

2.6.2 Optional

- **Fiat:** ECMWF Fiat library (set FIAT_DIR to enable)
-

2.7 Testing

2.7.1 Manual Testing

```
# Build
cd build && make

# Run a simple filter
cd /path/to/CCMA/database
odb_filter type 5

# Verify results
# (Check that only obstype=5 observations are active)
```

2.7.2 Adding New Tests

1. Create test SQL files in sql/
 2. Add test cases to src/filter.F90
 3. Document test scenarios in doc/examples.md
-

2.8 Contributing

2.8.1 Code Style

- **Fortran:** Follow existing indentation (2 spaces)
- **C:** Follow existing style (2 spaces, K&R braces)
- **CMake:** Use comments for major sections, keep variable names descriptive

2.8.2 Commit Messages

Follow conventional commit format: - **feat:** New features - **fix:** Bug fixes - **docs:** Documentation updates - **refactor:** Code refactoring - **build:** Build system changes

2.9 Debugging

2.9.1 Common Issues

“**odb98.x not found**” - Set `ODB_COMPILER` CMake variable - Or ensure `odb98.x` is in `PATH`

“**Schema not detected**” - Check `ddl.*` symlinks exist - Run `cmake ..` in build directory to re-detect

Link errors - Verify ODB libraries are in `pack` directory - Check `PACK` and `TYPE_PACK` settings

2.9.2 Debug Output

The `codb_set_entrypoint_()` function prints debug info when `myproc == 1`:

```
codb_set_entrypoint_(dbname='CCMA', dbname_len=4)
```

Enable by setting `myproc = 1` in the source code.

3 odbfilter Technical Reference

3.1 ODB (Observations Data Base)

3.1.1 Overview

ODB (Observations Data Base) is a database system used in numerical weather prediction for storing and processing observation data.

3.1.2 Key Concepts

- **Database Schemas:** Logical groupings of observation data (CCMA, ECMA)
- **Tables:** `hdr` (header), `body` (observation data), `errstat` (error statistics)
- **Views:** SQL-based queries that define data selection/modification
- **anchors:** Entry points for ODB to initialize schema functions

3.1.3 ECMA/CCMA Schema

Conventional and Satellite Observation Data

- Primary schema for conventional observations (TEMP, SYNOP, etc.) and satellite data
- Tables: `hdr`, `body`, `errstat`, `cma` (CMA-specific)
- Used by `odb_filter` as the main input/output database

- Used for validation and comparison
- Data can be transferred between CCMA and ECMA via shuffle operations

3.2 SQL Files

3.2.1 Location

All SQL files are stored in `sql/` directory. They are symlinked via `ddl.*` directories.

3.2.2 File Structure

Each SQL file follows this pattern:

```
//
//-- ODB/SQL file 'filename.sql'
//

UPDATED; // Indicates the view can modify data

SET $variable = default_value; // Optional variable declarations

CREATE VIEW view_name AS
  SELECT column1, column2 UPDATED, // UPDATED = modifiable
  FROM table1, table2
  WHERE condition=$variable
;
```

3.2.3 Key Keywords

Keyword	Purpose
UPDATED	Marks the view as capable of modifying data
SET \$var = val	Declares a variable with default value
CREATE VIEW	Defines the SQL view
UPDATED (column)	Marks a column as modifiable
\$variable	References a declared variable

3.2.4 Available SQL Files

3.2.4.1 Selection Views

File	Purpose	Variables
<code>select_all.sql</code>	Select all observations	-

File	Purpose	Variables
select_obstype.sql	Filter by observation type	\$otype
select_timeslot.sql	Filter by timeslot	\$tslot
select_obstype_vno.sql	Filter by type and variable	\$otype, \$vno
select_obstype_vno_ts.sql	Filter by type, variable, timeslot	\$otype, \$vno, \$tslot
select_obstype_wind.sql	Filter wind observations	\$otype, \$vno_u, \$vno_v
select_wind_obstype_vno_ts.sql	Wind with all params	\$otype, \$vno_u, \$vno_v, \$tslot
select_scat_ts.sql	Scatterometer by timeslot	\$tslot
select_obstype_sensor.sql	Filter by type and sensor	\$otype, \$sensor
select_onestatid.sql	By station ID	\$statid
select_oneobsare.sql	By type and area	\$otype, \$varno, \$lonmin/max, \$latmin/max, \$presmin/max
select_oneobs_rad.sql	Single radar observation	\$otype, \$vno, \$ident, \$distance, \$elev, \$azimut
select_oneobs_gps.sql	Single GPS sol observation	\$otype, \$ctype, \$lon, \$lat
select_oneobs_can.sql	By channel (canal)	\$otype, \$vertco
select_amsua.sql	AMSU-A observations	-
select_sophie.sql	Custom selection for Sophie	-
select_for_debug.sql	Debug selection	-

3.2.4.2 Update/Modify Views

File	Purpose
hdr_set_passive.sql	Set REPORT_STATUS.ACTIVE=0 (passive)
update_hdr_status.sql	Set REPORT_STATUS.ACTIVE=1 where DATUM_STATUS.ACTIVE=1
copy_fg_depar.sql	Copy first guess departures to column 2
copy_an_depar.sql	Copy analysis departures to column 2
delete_passive.sql	Delete passive observations
select_fg_actual_depar.sql	Select FG/actual departures

3.3 ODB Macros and Functions

3.3.1 In C Code (Generated)

3.3.1.1 ODB_ANCHOR(schema) Initializes the anchor point for a schema. Must be called before registering views.

```
ODB_ANCHOR(CCMA); // Initialize CCMA schema anchor
```

3.3.1.2 ODB_ANCHOR_VIEW(schema, view) Registers a SQL view function with ODB. Called within `${schema}_static_init()`.

```
ODB_ANCHOR_VIEW(CCMA, select_all); // Register select_all view for CCMA
```

3.3.1.3 Static_Init(db) Macro Defined in `_odb_glue.c`, this macro: 1. Checks if the provided `dbname` matches the schema 2. If matched, registers the schema's `db_static_init()` function

```
// Expanded macro for CCMA:
if (strcmp(dbname, "CCMA", dbname_len) == 0) {
    extern void CCMA_static_init();
    ODB_add2funclist("CCMA", CCMA_static_init, 0);
}
```

3.3.2 ODB API Functions

3.3.2.1 codb_set_entrpoint(dbname, dbname_len) Entry point called by ODB to set up database schema initializers.

- **Parameters:**
 - `dbname`: The database schema name (e.g., “CCMA”, “ECMA”)
 - `dbname_len`: Length of the `dbname` string (Fortran hidden argument)
- **Purpose:** Calls `Static_Init()` for each detected schema

3.3.2.2 codb_procdta(myproc, nproc, pid, it, inumt) ODB internal function to get processor information.

- **Parameters:** All are output parameters (pointers)
 - `myproc`: Current processor number
 - `nproc`: Total number of processors
 - `pid`: Process ID
 - `it`: Thread number
 - `inumt`: Number of threads

3.3.2.3 ODB_add2funclist(dbname, func, funcno) Registers a function in ODB's function list.

- **Parameters:**
 - `dbname`: Database schema name

- **func**: Function pointer to register
 - **funcno**: Function number (0 for static init)
-

3.4 Build Integration

3.4.1 odb98.x SQL Compiler

The odb98.x tool converts .sql files to C object files.

Command (from sql/CMakeLists.txt):

```
odb98.x -c -I${INC_CMA}.${cma} -l ${cma} -w -l ${cma} -w ${sql}
```

- **-c**: Compile mode
- **-I**: Include directory for schema headers
- **-l**: Schema name
- **-w**: Wrap function names
- **\${sql}**: SQL file to compile

3.4.2 Generated Files

3.4.2.1 \${schema}.h Generated by odb98.x during SQL processing. Contains: - Function declarations for each SQL view - Schema-specific definitions

Located in: \${CMAKE_BINARY_DIR}/ddl.\${schema}/\${schema}.h

3.4.2.2 \${schema}_Sstatic.c Generated from cmake/templates/_Sstatic.c.in. Contains: - \${schema}_static_init() function - ODB_ANCHOR() call - ODB_ANCHOR_VIEW() calls for each SQL view

Located in: \${CMAKE_BINARY_DIR}/src/\${schema}_Sstatic.c

3.4.2.3 _odb_glue.c Generated from cmake/templates/_odb_glue.c.in. Contains: - codb_set_entrypoint() function - Static_Init() macro definition - Calls to each schema's Static_Init()

Located in: \${CMAKE_BINARY_DIR}/src/_odb_glue.c

3.4.3 Object Libraries

For each schema, a static library is created:

- **\${schema}_lib**: Contains object files from all SQL views
 - Created in sql/CMakeLists.txt
 - Linked into the final executable
-

3.5 Environment Variables

3.5.1 ODB Runtime Variables

Variable	Purpose	Set In
ODB_CONSIDER_TABLES	Tables to consider (excludes /bufr/)	odb_filter_mod.F90
ODB_SRCPATH_CCMA	Source path for CCMA tables	odb_filter_mod.F90
ODB_DATAPATH_CCMA	Data path for CCMA database	odb_filter_mod.F90
ODB_STATIC_LINKING	Enable static linking (1=on)	odb_filter_mod.F90
ODB_IO_METHOD	I/O method (1=default)	odb_filter_mod.F90
ODB_CMA	Database type (CCMA)	odb_filter_mod.F90

3.5.2 Build Environment Variables

Variable	Purpose	Set In
ODB_COMPILER	Path to odb98.x	build_for_pack.sh, CMake
ODB_COMPILER_FLAGS	Flags for odb98.x	build_for_pack.sh
ODB_CREATE_IOASSIGN	Path to create_ioassign script	build_for_pack.sh
ODB_SYSPATH	System path for ODB schemas	build_for_pack.sh
ODB_CC	C compiler for ODB	build_for_pack.sh

3.6 Database Tables

3.6.1 hdr (Header Table)

Contains metadata about each observation report.

Column	Description
expver	Experiment version
obstype	Observation type (5=TEMP, 4=SYNOP, etc.)
codetype	Code type
sensor	Sensor number
statid	Station ID
lat, lon	Latitude, Longitude
stalt	Station altitude

Column	Description
<code>date, time</code>	Observation date and time
<code>report_status</code>	Report status (active/passive)
<code>datum_status</code>	Datum status (active/passive)

3.6.2 body (Body Table)

Contains the actual observation data.

Column	Description
<code>varno</code>	Variable number (2=temp, 3=pressure, etc.)
<code>obsvalue</code>	Observation value
<code>obserr</code>	Observation error
<code>fg_depar</code>	First guess departure
<code>an_depar</code>	Analysis departure
<code>biascorr</code>	Bias correction

3.6.3 errstat (Error Statistics Table)

Contains error statistics for quality control.

3.7 Fortran-ODB Integration

3.7.1 Key Modules and Functions

3.7.1.1 odb_filter_mod Module

- `odb_filter_start(odb, filter, vars, vals, nb_obs):`
 - Initializes ODB environment
 - Parses filter-specific arguments
 - Opens CCMA database in READWRITE mode
 - Returns ODB handle, variables, and values
- `odb_filter_end(odb, ok):`
 - Updates `REPORT_STATUS.ACTIVE` based on `DATUM_STATUS.ACTIVE`
 - Closes ODB (commits if `ok!=0`, discards otherwise)

3.7.1.2 ODB Fortran API

- `ODB_select(odb, view, ...):` Selects data using a SQL view
- `ODB_get(...):` Gets data from a selection
- `ODB_put(...):` Puts (writes) modified data
- `ODB_close(odb):` Closes ODB handle

3.7.2 Special Values

Value	Meaning
rmdi	Real Missing Data Indicator (for Fortran reals)
1	Active observation
0	Passive observation
-1	Deleted observation

3.8 Troubleshooting

3.8.1 SQL Compilation Errors

Symptom: odb98.x fails to compile .sql files

Solution: 1. Check SQL syntax (missing semicolons, etc.) 2. Verify odb98.x is in PATH or set ODB_COMPILER 3. Check include paths (INC_CMA CMake variable)

3.8.2 Missing Headers

Symptom: "CCMA.h": No such file or directory

Solution: 1. Ensure odb98.x ran successfully (check build directory) 2. Verify \${CMAKE_BINARY_DIR}/ddl.CCMA/ is in include path 3. Re-run CMake to regenerate headers

3.8.3 Schema Not Found

Symptom: Static_Init() not called for a schema

Solution: 1. Check ddl.* symlinks exist: `ls -la ddl.*` 2. Verify symlinks point to sql/: `readlink ddl.CCMA` 3. Re-run CMake: `rm -rf build && mkdir build && cd build && cmake ..`

4 odbfilter Examples

4.1 Command-Line Usage

4.1.1 Basic Syntax

`odb_filter <filter_type> [arguments]`

All filters read from a CCMA database and modify observation statuses.

4.2 Filter Examples

4.2.1 1. Select Observations by Time Slot

Filter: timeslot

Purpose: Keep only observations from a specific time slot.

```
odb_filter timeslot 12
```

- Selects observations from time slot 12
 - All other observations are marked as passive
-

4.2.2 2. Keep Only Specific Observation Type

Filter: type

Purpose: Keep only observations of a given type, mark others as passive.

```
# Keep only TEMP observations (obstype=5)
```

```
odb_filter type 5
```

4.2.3 3. Delete Specific Observation Type

Filter: deletetype

Purpose: Delete all observations of a given type.

```
# Delete all TEMP observations
```

```
odb_filter deletetype 5
```

This sets `DATUM_STATUS.ACTIVE=0` and `REPORT_STATUS.ACTIVE=0`.

4.2.4 4. Filter by Observation Type and Variable

Filter: varno

Purpose: Keep observations matching both type and variable number.

```
# Keep TEMP temperature observations (varno=2)
```

```
odb_filter varno 5 2
```

4.2.5 5. Select Single Observation

Filter: singleobs

Purpose: Select a single observation of specified type/varno.

```
# Select one TEMP temperature observation
```

```
odb_filter singleobs 5 2
```

```
# Select one TEMP observation from timeslot 6
```

```
odb_filter singleobs 5 2 6
```

```
# Select 3 TEMP observations
```

```
odb_filter singleobs 5 2 6 3
```

Arguments: 1. `obstype`: Observation type 2. `varno`: Variable number 3. `[timeslot]`: Optional time slot 4. `[nb_obs]`: Optional number of observations to select (default: 1)

4.2.6 6. Select Wind Observations

Filter: wind

Purpose: Select wind observations (u and v components).

```
# Select wind observations for TEMP (obstype=5), varno_u=7
```

```
odb_filter wind 5 7 6 1
```

Arguments: 1. `obstype`: Observation type 2. `varno_u`: Wind u-component variable number 3. `timeslot`: Time slot 4. `partition`: Partition number

4.2.7 7. Filter by Geographic Area

Filter: singleobsarea

Purpose: Select observations within a geographic area.

```
# Select TEMP obs in area: lon -10 to 10, lat 40 to 50
```

```
odb_filter singleobsarea 5 2 -10 10 40 50
```

```
# Also filter by pressure: 950 to 1050 hPa
```

```
odb_filter singleobsarea 5 2 -10 10 40 50 950 1050
```

Arguments: 1. `obstype`: Observation type 2. `varno`: Variable number 3. `lonmin`, `lonmax`: Longitude range 4. `latmin`, `latmax`: Latitude range 5. `[presmin]`, `[presmax]`: Optional pressure range (hPa)

4.2.8 8. Filter by Station ID

Filter: singlestatid

Purpose: Select observations from a specific station.

```
# Select observations from station 12345
```

```
odb_filter singlestatid 12345
```

Arguments: 1. `statid`: Station ID

4.2.9 9. Filter Scatterometer Observations

Filter: `scat`

Purpose: Select scatterometer observations for a time slot.

```
# Select scatterometer obs from timeslot 6
```

```
odb_filter scat 6
```

```
# Select 5 scatterometer observations
```

```
odb_filter scat 6 5
```

Arguments: 1. `timeslot`: Time slot 2. `[nb_obs]`: Optional number of observations (default: 1)

4.2.10 10. Filter by Sensor

Filter: `sensor`

Purpose: Select observations from a specific sensor.

```
# Select TEMP observations from sensor 12
```

```
odb_filter sensor 5 12
```

Arguments: 1. `obstype`: Observation type 2. `sensor`: Sensor number

4.2.11 11. Select Single Radar Observation

Filter: `singleradar`

Purpose: Select a single radar observation with specific criteria.

```
# Select radar observation
```

```
odb_filter singleradar 5 2 "IDENT123" 100.5 500 45.0
```

Arguments: 1. `obstype`: Observation type 2. `varno`: Variable number 3. `ident`: Identifier (quoted string) 4. `distance`: Distance (km) 5. `elev`: Elevation (m) 6. `azimut`: Azimuth (degrees)

4.2.12 12. Select Single GPS Solution

Filter: singlegpsol

Purpose: Select a single GPS solution observation.

Select GPS observation at given position

odb_filter singlegpsol 17 76 2.5 48.8

Arguments: 1. obstype: Observation type (17=GPS) 2. codetype: Code type 3. lon: Longitude 4. lat: Latitude

4.2.13 13. Filter by Channel (Canal)

Filter: singleobsanal

Purpose: Select observations by satellite channel.

Select AMSU-A channel 5

odb_filter singleobsanal 4 2 5

Arguments: 1. obstype: Observation type 2. vertco: Vertical coordinate (channel number)

4.2.14 14. Copy First Guess Departures

Filter: fg_depar

Purpose: Copy first guess departures to a different column.

odb_filter fg_depar

Modifies the database by copying fg_depar values to column 2.

4.2.15 15. Copy Analysis Departures

Filter: an_depar

Purpose: Copy analysis departures to a different column.

odb_filter an_depar

Modifies the database by copying an_depar values to column 2.

4.2.16 16. Delete Passive Observations

Filter: delete_passive

Purpose: Permanently delete all passive observations.

odb_filter delete_passive

Removes observations where DATUM_STATUS.ACTIVE=0.

4.2.17 17. Debug Filter

Filter: debug

Purpose: Run a debug selection for troubleshooting.

odb_filter debug

Uses sql/select_for_debug.sql for special debugging criteria.

4.3 Using Wrapper Scripts

4.3.1 odb_filter.sh

Sets up ODB environment and runs odb_filter:

```
# Basic usage
```

```
scripts/odb_filter.sh type 5
```

```
# With shuffle
```

```
scripts/odb_filter.sh singleobs 5 2
```

This script also: 1. Performs shuffle (CCMA → ECMA) 2. Copies first guess departures (odb_filter fg_depar)

4.3.2 odbtbx.sh (Toolbox)

Provides filter and shuffle functions:

```
# Apply filter
```

```
scripts/odbtbx.sh -f type 5
```

```
# Perform shuffle
```

```
scripts/odbtbx.sh -s
```

```
# Both filter and shuffle
```

```
scripts/odbtbx.sh -f singleobs 5 2 -s
```

Options: - **-f**: Apply specified filter - **-s**: Perform shuffle operation

4.3.3 CCMA2ECMA

Converts a CCMA database to pseudo-ECMA format:

```
# In a directory containing CCMA/  
scripts/CCMA2ECMA
```

This script: 1. Renames CCMA/ to ECMA/ 2. Filters IOASSIGN file 3. Replaces “CCMA” with “ECMA” in file contents 4. Recreates IOASSIGN symlinks

4.4 Advanced Scenarios

4.4.1 Batch Processing Multiple Filters

```
# Process multiple time slots  
for ts in 1 2 3 4 5 6; do  
    odb_filter timeslot $ts  
done
```

4.4.2 Combining Filters

```
# First select by type, then by area  
odb_filter type 5  
odb_filter singleobsarea 5 2 -10 10 40 50
```

4.4.3 Working with Wrapper Scripts in Production

```
# Set up environment  
export ODB_CMA=CCMA  
export ODB_SRCPATH_CCMA=/path/to/odb/src/main/odb/ddl.CCMA  
export ODB_DATAPATH_CCMA=/path/to/data  
  
# Run filter with shuffle  
scripts/odbtbx.sh -f singleobs 5 2 -s
```

4.5 Verifying Results

4.5.1 Check Active Observations

After running a filter, verify the results:

```
# Use ODB tools to check  
odbsql "SELECT obstype, varno, COUNT(*) FROM hdr, body WHERE datum_status.active=1 GROUP BY
```

4.5.2 Check Report Status

Check report_status values

```
odbsql "SELECT report_status.active, COUNT(*) FROM hdr GROUP BY report_status.active"
```

4.6 Common Patterns

4.6.1 Keep Only One Observation Type

```
odb_filter type 5 # Keep only TEMP
```

4.6.2 Select Specific Observations

```
odb_filter singleobs 5 2 6 # One TEMP temp obs from timeslot 6
```

4.6.3 Filter by Area

```
odb_filter singleobsarea 5 2 -10 10 40 50 # TEMP in area
```

4.6.4 Work with Wind

```
odb_filter wind 5 7 6 1 # TEMP wind from timeslot 6
```
